DDSS Project Instructions

1. Role and Goal

You are a senior Python developer and DSA teacher. Build a complete university project called **DDSS - Dawood Data Science Society Members & Volunteer Registration System** for the Department of Data Science, Dawood University of Engineering and Technology (DUET). It must be a **web application with a Python frontend and a Python backend**. Create the whole project (code, sample data, requirements, README and a short project report), run it, test it, and fix every error before finishing.

The system manages society members, volunteers, events, and event registrations. It must also **demonstrate practical use of Data Structures and Algorithms (DSA)**, so the data structures and algorithms must be written by hand (section 6).

2. Logos and Branding

- Use the two logos in **assets/duet_logo.png** (DUET official logo) and **assets/ddss_logo.png** (Data Science Society logo).
- Show **both logos side by side at the top of every page** (DUET on the left, DDSS on the right, with the title in the middle), and also on the login page and in the sidebar.
- Use the page title "DDSS - Dawood Data Science Society" and the tagline "Inspire - Analyze - Innovate".
- Use the logo colours (blue and black on white) for the theme. Make the layout clean, responsive and professional.

3. Architecture and Technology Rules

- Language: **Python 3.10+** for everything.
- **Backend:** FastAPI (run with uvicorn) on port 8000. It holds all business logic, the DSA structures, validation, authentication and Excel read/write, and exposes REST endpoints.
- **Frontend (webpage):** Streamlit on port 8501. It only shows pages and forms and talks to the backend using the requests library. No business logic in the frontend.
- Storage: **Excel workbook (records.xlsx)** using openpyxl (pandas only for showing tables or exporting), plus **activity_log.txt**, plus an **uploads/** folder for ID card pictures. No SQL database.
- Allowed libraries: fastapi, uvicorn, python-multipart, streamlit, requests, openpyxl, pandas, pillow, plus the Python standard library. List them in requirements.txt.
- Use a threading lock around every Excel write so two submissions at the same time cannot corrupt the file.
- Do NOT use list.sort(), sorted() or dict lookups as a replacement for the required algorithms; write them yourself (section 6).
- Provide **run_all.py** (or run_all.bat) that starts the backend and the frontend together, and also document how to start them in two terminals.

4. Roles, Login and Dashboards

Role	Access
Admin (1 account)	Full control and access to ALL data. Admin dashboard (below).
Cabinet members (9 accounts)	Each has a personal login and dashboard (below). Cannot manage accounts or delete data.

Role	Access
Student (no login)	Can only open an event form that is currently OPEN and submit it. Identified by roll number.

- **Authentication:** username and password for admin and cabinet members. Store passwords **hashed** (hashlib.pbkdf2_hmac with a salt), never in plain text. Use a token or session returned by the backend; every protected endpoint must check the role. Provide Logout. Pre-create 1 admin and 9 cabinet accounts with default passwords shown in the README, and let the admin reset a password.
- **Cabinet roles:** create 9 cabinet members with editable titles (for example President, Vice President, General Secretary, Treasurer, Event Manager, Media Head, Technical Head, Marketing Head, Discipline Head). I will replace these with the real names later through the admin dashboard.

Admin dashboard (full access)

- Overview: total members, volunteers, events, forms (open and closed), registrations (pending, approved, rejected) and attendance, with simple charts.
- **Form management:** create, edit, open, close and delete forms (section 5).
- **All data:** view, search, sort, edit, approve/reject and delete every record in every sheet (members, volunteers, attendees, events, attendance, assignments, cabinet accounts).
- View each submitted ID card picture and download any file.
- Manage the 9 cabinet accounts (add, edit, deactivate, reset password) and the allowed roll number list (upload a CSV).
- Download **records.xlsx**, the ID card images (zip) and **activity_log.txt**, and view the audit log. Undo last action (stack).

Cabinet member dashboard (each of the 9 members)

- Personal dashboard showing the member's name and title and the society logos.
- View events, open/closed forms, volunteer and attendee submissions, and statistics.
- Search, sort and filter registrations; approve or reject volunteer and attendee requests (processed through the queue); mark attendance.
- Download reports (CSV or Excel) for events. Cannot delete data, change forms' open/close state, or manage accounts (admin only).
- Every cabinet action is written to the activity log with the member's name.

5. Registration Forms (Two Types, Opened and Closed by Admin)

There are **two form types**, each created by the admin for a specific event:

- **Form 1 - Volunteer Form** (students who want to volunteer for the event).
- **Form 2 - Attendee Form** (students who want to attend the event).

Admin controls for each form

- **Form title** = event title, and a **description** text that the admin types when creating the form. Both are shown at the top of the student form.
- **Open and close:** the admin can open or close the form manually with one click, AND can set an **open date and time** and a **close date and time** so the form opens and closes automatically at the right time. The backend checks the current time on every request, so no background scheduler is needed.
- A closed or not-yet-open form must show a clear message ("This form is closed" or "Opens on ...") and the backend must reject submissions to it.

- Optional: maximum number of seats. When full, the form shows "Registrations full".

Student form fields (same structure for both forms)

Field	Rules
Candidate Name	Required text, letters and spaces only, 3 to 60 characters.
Roll No	Required. Format like **25F-DS-052** (batch-course-serial, see section 7). Must be in the allowed list. One submission per roll number per form.
Phone No	Required. Valid Pakistani mobile number (for example 03XXXXXXXXX or +923XXXXXXXXX).
Department	Required. Dropdown (Data Science and the other departments).
ID Card Picture	**Mandatory.** JPG, JPEG or PNG only. **Maximum 500 KB** (512000 bytes). Check size and real image type in the frontend AND the backend (use Pillow to confirm it is a valid image). Reject larger or invalid files with a clear error message.

- After a successful submission show a confirmation with a reference number. Save the picture as **uploads/<form_id>/<roll_no>_idcard.<ext>** and keep the file name in the Excel row. ID card pictures are private: only admin and cabinet members may see them.
- All submissions go into the queue for processing (section 6) with the status Pending, then Approved or Rejected by admin or cabinet.

6. Data Structures and Algorithms (Core Marks)

Implement these yourself in **dsa.py** (no collections.deque and no built-in queue or stack modules). Each one must be used by a real feature:

Structure	Used for	Operations
Singly Linked List	Member and volunteer records	insert at end, delete by ID, update, traverse, find by ID
Array (Python list used as an array)	Event and form schedule kept sorted by date	append, index access, binary search
Queue (own class, FIFO)	Form submissions waiting for approval, processed in arrival order	enqueue, dequeue, peek, is_empty, size
Stack (own class, LIFO)	Undo last action (add, update, delete)	push, pop, peek, is_empty, size
Linked list per member	Participation history	add event, traverse

Algorithms

- **Searching:** linear search (by name, department, partial text) and **binary search** (by roll number or ID on sorted data). Show which algorithm was used and the number of comparisons.
- **Sorting:** implement at least **bubble or insertion sort** AND **merge sort or quick sort**. Let the user choose the algorithm and the key (name, roll number, date, timestamp). Show the algorithm name and the time taken.
- Add a comment above each function with its time complexity (for example binary search O(log n), merge sort O(n log n)).

7. Student-Only Rule (Roll Numbers)

Only university students of the Data Science department may submit forms. A roll number looks like **25F-DS-052**: **25F** = batch, **DS** = course, last part = serial number (2 or 3 digits).

```
import re
ROLL_PATTERN = re.compile(r"^\d{2}F-DS-\d{2,3}$")
```

```
def valid_roll_no(roll: str) -> bool:
    return ROLL_PATTERN.fullmatch(roll.strip().upper()) is not None
```

- Convert to uppercase and trim spaces before checking. Also check the roll number against **allowed_roll_numbers.csv** (pre-fill it with 25F-DS-052, 25F-DS-076, 25F-DS-085, 25F-DS-087, 25F-DS-093 and sample numbers 25F-DS-001 to 25F-DS-025). Show a clear error for invalid or unlisted roll numbers.
- Reject duplicate roll numbers for the same form.

8. Excel Records and Log (Teacher Rechecking)

- **All data is saved in records.xlsx**, created automatically, with a header row on each sheet: **Members, Volunteers, Attendees, Events, Forms, Registrations, Attendance, Assignments, Cabinet**.
- Volunteers and Attendees sheets use the columns: Submission ID, Form ID, Event Title, Candidate Name, Roll No, Phone No, Department, ID Card File, Submitted At, Status, Reviewed By.
- Save after every submission, edit, delete, approval or attendance action. Load the workbook when the backend starts so data stays after restart. Never wipe or overwrite existing data when the app restarts.
- **activity_log.txt**: one line per action, in the format *2026-10-24 11:15:03* | *25F-DS-052* | *SUBMIT_VOLUNTEER_FORM* | *Form F01 submitted*. Include login, logout, failed login, form open/close, submissions, approvals, edits, deletes, undo and errors.
- Handle an Excel file that is open or locked with a clear message instead of crashing.

9. Project Structure

```
ddss_project/
  assets/duet_logo.png        # DUET official logo
  assets/ddss_logo.png        # Data Science Society logo
  backend/main.py             # FastAPI app and routes
  backend/auth.py             # password hashing, tokens, roll number check
  backend/storage.py          # records.xlsx, activity_log.txt, uploads/
  backend/dsa.py              # LinkedList, Stack, Queue, search and sort
  backend/models.py           # Member, Event, Form, Submission classes
  frontend/app.py             # Streamlit entry (public forms + login)
  frontend/pages/             # admin dashboard, cabinet dashboard
  allowed_roll_numbers.csv
  records.xlsx                # created automatically
  activity_log.txt            # created automatically
  uploads/                    # ID card pictures (created automatically)
  run_all.py
  requirements.txt
  README.md
  PROJECT_REPORT.md
  tests/test_dsa.py           # tests for stack, queue, linked list, search, sort
```

10. Quality, Testing and Deliverables

- Validate every input in the backend and show friendly messages in the frontend. The app must never crash on empty input, duplicates, wrong IDs, a bad file or an empty list.
- Add sample data: 10 members, 5 volunteers, 5 events, 2 open forms and 1 closed form, and the 10 default accounts, so the project can be demonstrated immediately.
- Write tests in tests/test_dsa.py (stack, queue, linked list, binary search, both sorts) and API tests for: form closed, file larger than 500 KB, wrong file type, invalid roll number, duplicate submission, cabinet trying an admin-only action. Run them and fix failures.

- Start the backend and frontend, click through the admin dashboard, a cabinet dashboard and both student forms, and fix all errors before finishing.
- **README.md:** installation, how to run, default admin and cabinet logins, test roll numbers, and a feature list.
- **PROJECT_REPORT.md:** a short report explaining (a) why each data structure was chosen for each feature, (b) the time complexity of each algorithm, (c) the architecture (frontend, backend, Excel), (d) limitations and future improvements. Write it so a student can explain it in a viva.
- At the end, give a summary of what was built and the exact commands to run it: **pip install -r requirements.txt** then **python run_all.py**.